Multilingual ASR Pipeline

Automated Speech Transcription, Speaker Identification & Translation

Sayouti Noureini

Work in Progress

February 19, 2026

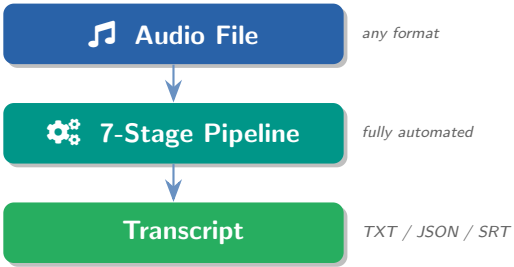
Outline

What Does the Pipeline Do?

Given an **audio recording** in **any language**, the pipeline automatically produces:

A **professional transcript**

- **Speaker identification** — who said what
- **Precise timestamps** for every utterance
- **English translation** of all content
- **Cleaned-up text** — ready for analysis



Key Strength

Supports **1,600+ languages**, including low-resource languages often ignored by commercial tools.

Hardware Specifications

Development Machine

GPU	NVIDIA — **6 GB VRAM**
CPU	Multi-core
RAM	16 GB system
OS	WSL2 on Windows 11
Runtime	Python 3.12 + CUDA

The 6 GB Constraint

Most ASR pipelines assume 16–24 GB VRAM. With only **6 GB**, every decision is driven by **memory efficiency**.

How the Pipeline Adapts

- **Sequential model loading** — one model on GPU at a time
- **Explicit unloading** — `load()/unload()` lifecycle
- **4-bit quantization** — TranslateGemma 4B in ∼2.5 GB
- **GPU memory flush** — `empty_cache()` + `gc.collect()`
- **float16 compute** — halves memory vs. float32
- **CTC 300M** over 1B — fits in 6 GB with room to spare

Performance on 6 GB GPU

Current Benchmark

8 minutes processing time

for a **20-minute** audio file

Real-Time Factor: **0.4**×
(faster than real-time)

Real Test (Spanish, 20 min)

Engine	Whisper large-v3
Speakers	5 identified
Speech	61% speech, 39% non-speech
Translation	TranslateGemma
Time	**8 min** end-to-end

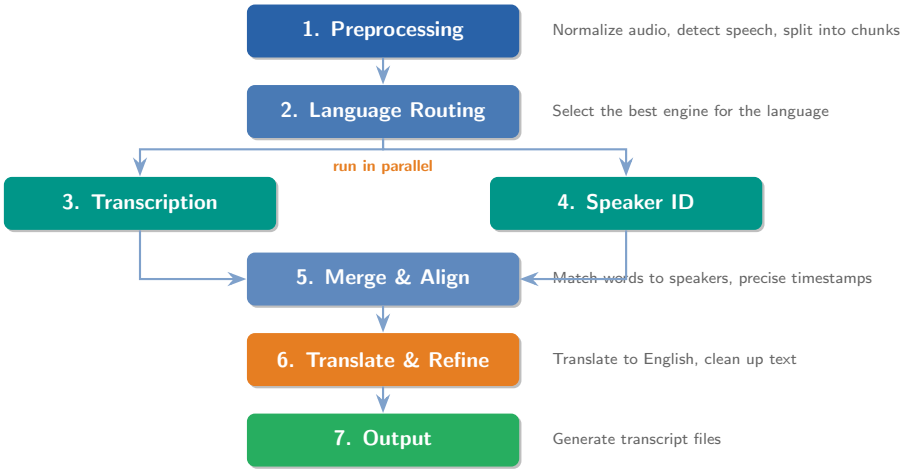
Speed Optimizations

1. **Parallel execution** — transcription & diarization run concurrently
2. **Batched inference** — Whisper batched pipeline (batch_size=8)
3. **VAD-guided chunking** — skips silence, only processes speech
4. **Lazy model loading** — models load on first use
5. **Spectral noise reduction** — cleaner audio = faster ASR

Memory Choreography

ASR models unloaded *before* translation loads. Peak VRAM never exceeds ∼4 GB at any stage.

The 7-Stage Pipeline



Stage 1: Audio Preprocessing

What Happens

1. **Format conversion** — any format to WAV (16 kHz, mono)
2. **Loudness normalization** — consistent volume (EBU R128)
3. **Noise reduction** — spectral gating
4. **Voice detection** — separates speech from silence/noise/music
5. **Chunking** — splits long recordings into segments

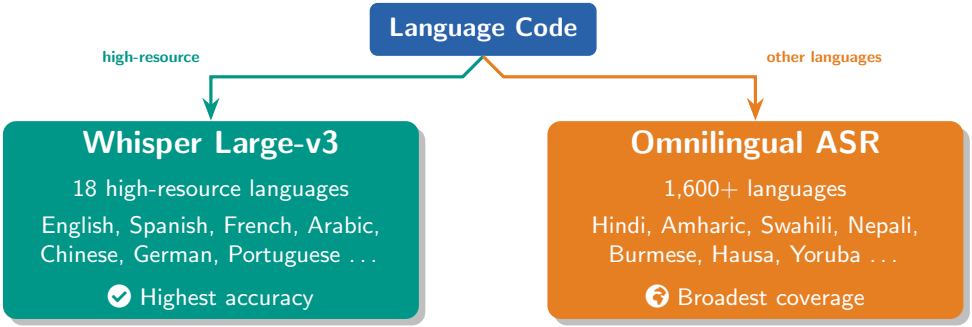
Why It Matters

- Handles **any input format** (MP3, M4A, WAV, etc.)
- Improves accuracy by **reducing noise**
- **Audio quality metrics** in output (speech ratio, silence gaps)
- Non-speech classified as **silence**, **noise**, or **music**

Tools Used

FFmpeg, Silero VAD, Spectral Gating

Stage 2: Intelligent Language Routing



The pipeline automatically selects the **best-performing engine** for each language.
Adding a new language requires **only one line of configuration**.

Stages 3 & 4: Transcription + Speaker Identification

Stage 3: Transcription

Converts speech to text

- **Whisper Large-v3** for high-resource languages
  - State-of-the-art accuracy
  - Word-level timestamps
- **Omnilingual CTC 300M** for all others
  - Meta's multilingual model
  - Native code-switching support

Stage 4: Speaker Identification

Detects who speaks when

- Auto-determines **number of speakers**
- Labels: `SPEAKER_00`, `SPEAKER_01` ...
- Works **independently** of language
- Can override speaker count if known

Two Backends

pyannote 3.1 (default) or **NeMo MSDD** (overlap-aware)

These two stages run **in parallel** on the GPU, saving significant processing time.

Stages 5–6: Alignment, Translation & Refinement

Stage 5: Merge & Align

- Matches each **word to its speaker** via temporal overlap
- **Forced alignment** (wav2vec2 MMS) refines timestamps
- Whisper: ±200ms–1s drift → after alignment: ∼**20ms**
- Same-speaker segments **merged** into natural turns

Stage 6: Translate & Refine

TranslateGemma 4B (default):

- Single model: translation + text cleanup
- **4-bit quantized** → ∼2.5 GB VRAM
- 55 languages supported

Alternative: CT2 NLLB + Ollama LLM

- NLLB-200 for translation (200+ languages)
- LLM for refinement (Qwen 2.5:7b)

ASR/diarization models are **fully unloaded** before translation loads — peak VRAM stays within 6 GB.

Stage 7: Output

Real Output — Spanish Test (excerpt)

```
======================================
TRANSCRIPT
======================================
Duration:    00:20:00
Languages:   Spanish
Speakers:    5 identified
ASR Engine:  Whisper large-v3
Audio:       61% speech | 39% non-speech
======================================
        [NOISE -- 7.2s]
[00:00:07] SPEAKER_00:
[es] Y como le he comentado,
     queremos comprender mejor los
     tipos de cuidado infantil...
[en] As I mentioned earlier, we want
     to better understand the types
     of childcare provided...
[00:00:20] SPEAKER_00:
[es] Si gusta les dejo la cartita
     de presentacion de la empresa.
[en] If you like, I will include the
     company introductory letter.
======================================
```

Output Formats

- **TXT** — Research transcript with metadata
- **JSON** — Structured data for analysis
- **SRT** — Subtitles for video
- **All** — Generate all at once

Transcript Includes

- Full metadata header
- Speaker labels per turn
- Original text + English translation
- Non-speech markers (silence, noise, music)
- Audio quality metrics

Language Coverage & Adding New Languages

High-Resource (Whisper)

- English
- Spanish
- French
- German
- Portuguese
- Russian
- Chinese
- Japanese
- Korean
- Italian
- Dutch
- Polish
- Turkish
- Czech
- Swedish
- Ukrainian
- Romanian
- Arabic

Low-Resource (Omnilingual)

- Hindi
- Bengali
- Nepali
- Swahili
- Amharic
- Oromo
- Hausa
- Yoruba
- Igbo
- Tagalog
- Burmese
- Khmer
- Kinyarwanda
- Somali
- Tigrinya
- *+ 1,585*

Add a Language

One YAML entry — no retraining:

```
wol:
  name: "Wolof"
  tier: "non_high"
  bcp47: "wo"
  script: "Latn"
  nllb_code: "wol_Latn"
```

Code-Switching

Handles mid-sentence language switching automatically.

Zero-Shot

Unseen languages: auto-fallback to LLM variant.

Fine-Tuning for New Languages

Why Fine-Tune?

- Out-of-the-box: CER <10% for **78%** of 1,600+ languages
- Fine-tuning pushes accuracy **significantly higher**
- Useful for **dialectal variation** or **domain-specific** vocab

Straightforward Process

1. Prepare audio + transcription pairs
2. Use Meta's fine-tuning recipes (wav2vec2 CTC)
3. CTC 300M fits on a **single 6 GB GPU**
4. Point pipeline config to new checkpoint

Pipeline Integration

`LanguageConfig` supports custom checkpoints:

```
class LanguageConfig:
    name: str
    tier: LanguageTier
    bcp47: str
    script: str
    nllb_code: str
    fine_tuned_checkpoint: Optional[str]
```

Register in YAML:

```
wol:
  name: "Wolof"
  tier: "non_high"
  fine_tuned_checkpoint:
    "./models/wolof_ctc300m.pt"
```

Recipes: `github.com/facebookresearch/omnilingual-asr`

How to Use It

Basic Usage

```
# Transcribe a Spanish interview
asr-pipeline transcribe interview.mp3 --language spa
# Hindi focus group with 5 speakers, JSON output
asr-pipeline transcribe focus_group.wav --language hin \
    --max-speakers 5 --format json
# Amharic with project name
asr-pipeline transcribe meeting.m4a --language amh \
    --project "Ethiopia Study" --output-dir ./transcripts
```

Other Commands

`list-languages` — show languages
`check-deps` — verify install
`setup` — configure models

Configuration

All settings via **YAML config**: audio, engine selection, output format, hardware.

Next Steps

Done

- Full 7-stage pipeline, end-to-end
- 1,600+ language support
- 8 min for 20 min audio on 6 GB GPU
- Speaker diarization (2 backends)
- Translation + refinement
- Multiple output formats (TXT, JSON, SRT)
- Forced alignment for precise timestamps
- Memory-efficient GPU management

Future Possibilities

- Web interface for non-technical users
- Real-time transcription mode
- Custom vocabulary support

Immediate Next Steps

1. **Single-file quality assessment**
   Run pipeline on one audio, review transcript accuracy, speaker assignment, translation quality.
2. **Batch processing (ZIP input)**
   Accept a **ZIP folder** of audio files, produce a folder of transcripts. Output files **match input filenames** for easy pairing with survey/para-data.
3. **Language-specific fine-tuning**
   Fine-tune CTC 300M on project languages to improve accuracy beyond baseline.

Thank You

Questions & Discussion

```
>_ asr-pipeline transcribe audio.mp3 --language spa
```